not done. You could write the last line as expect(project.done?).not_to be_truthy, but that seems harder to read. (You'll see an even shorter way to write that line in a little bit.)

The first failure is that the Task constant is missing. You can clear it up easily:

basics/02/app/models/task.rb
```
class Task
end
```

At this point, the next failure is that project.tasks doesn't exist. You can clear that up, and then also clear up the following failure, that project.done? doesn't exist. This leads you to write simple, but still incomplete, done? logic to make the test pass—a project is done if it has no tasks:

basics/02/app/models/project.rb
```
class Project
  attr_accessor :tasks

  def initialize
    @tasks = []
  end

  def done?
    tasks.empty?
  end
end
```

The second test now passes. And you have a clear candidate for the next test—the distinction between complete and incomplete tasks. I'm also starting to think about how you get from that distinction to the more advanced logic of sizing tasks and estimating project due dates.

With the test passing, you now enter the refactoring phase.

If you've done Rails programming before, you may have noticed that I have not yet made Project and Task subclasses of ActiveRecord::Base, meaning I haven't connected them to the database via ActiveRecord.

I have two reasons for this. The first is a purist one—I haven't added any tests that would need ActiveRecord functionality to pass. The second is a teaching-specific one—for the first testing example, I'd like to focus on the testing part, not the Rails boilerplate. Getting caught up in generating migrations and the like is not what I want to talk about here.

And, while there's a pretty good chance that both these classes will become ActiveRecord classes, it's not guaranteed, and if you're using tests to drive design it makes sense not to let the design leap ahead of the tests, but rather to use the tests to suggest the code's structure.